Amortized Post-Quantum Secure Aggregation for Federated Learning

Author Name
Affiliation
email

***Abstract*—**

***Index Terms*—post-quantum cryptography, federated learning, ML-KEM, ML-DSA, secure aggregation**

I. Introduction

II. Background

A. Post-Quantum KEMs and Signatures

B. Federated Learning and Update Confidentiality

III. Threat Model

IV. Protocol Design

A. Session Setup

B. Per-Round Update Encryption

C. Re-Keying Schedule

D. Operating Modes

V. Implementation

VI. Experimental Setup

VII. Results

A. Per-Round and Cumulative Crypto Overhead

B. Communication Overhead

C. Effect of Re-Key Interval

D. Scaling with the Number of Clients

E. Model Quality

VIII. Security Evaluation

IX. Discussion and Limitations

X. Related Work

XI. Conclusion

Appendix A
Implementation Notes

- **KEM input lengths unchecked by liboqs-python.** `encap_secret()` accepted a 1,183-byte ML-KEM-768 public key (expected 1,184) without error. *Fix:* wrapper validates public-key and ciphertext lengths; malformed keys are rejected as authentication failures.
- **MITM on the client-only-signed handshake.** An unauthenticated KEM ciphertext let an attacker re-encapsulate and learn the client's session key. *Fix:* server signs its response, bound to $H(init)$, in every mode.
- **Header epoch not covered by the AEAD.** With server-side AAD, a changed header epoch still authenticates; mutation M5 showed the explicit header check is the only defense. *Fix:* check retained and tested.
- **Cold-start timing bias.** First X25519 keygen ≈3.8 ms vs. ≈31 µs warm (lazy OpenSSL init) inflated classical round 0 (8.8 vs. 4.0 ms). *Fix:* untimed warm-up of all primitives before round 0.
- **Participation-dependent epochs.** Counting handshakes would desynchronize clients that skip rounds. *Fix:* scheduled epoch $e = \lfloor r/N \rfloor$; stale or non-increasing handshake epochs rejected.
- **Cadence-confounded baseline.** Classical originally keyed once. *Fix:* classical supports the same N, isolating the primitives.
- **Training cost.** A full MNIST pass took ≈13 s per round (about 9 h for the grid). *Fix:* per-client sample cap (2,000) and a run cache shared across experiments (80 distinct runs instead of 105).
- **Library/binding version coupling.** liboqs and liboqs-python must share major.minor; a missing library triggers a source build. *Fix:* liboqs 0.16.0 (Homebrew) with liboqs-python 0.16.0.1 pinned.
- **Interpreter choice.** System default Python 3.14. *Fix:* virtualenv on Python 3.11 for torch/torchvision wheel availability.
- **Manifest recorded CLI flags, not effective settings.** *Fix:* effective seeds, rounds, K, N, sweeps and sample cap stored and consumed by the asset builder.
- **Output naming.** Naive PQC CSVs were labelled N=none. *Fix:* names use the effective interval.
- **Benchmark conditions.** Laptop on battery at run time. *Fix:* full run deferred to AC power; runs executed sequentially.
- **No TeX toolchain.** *Fix:* `tectonic` installed for diagrams and the skeleton.

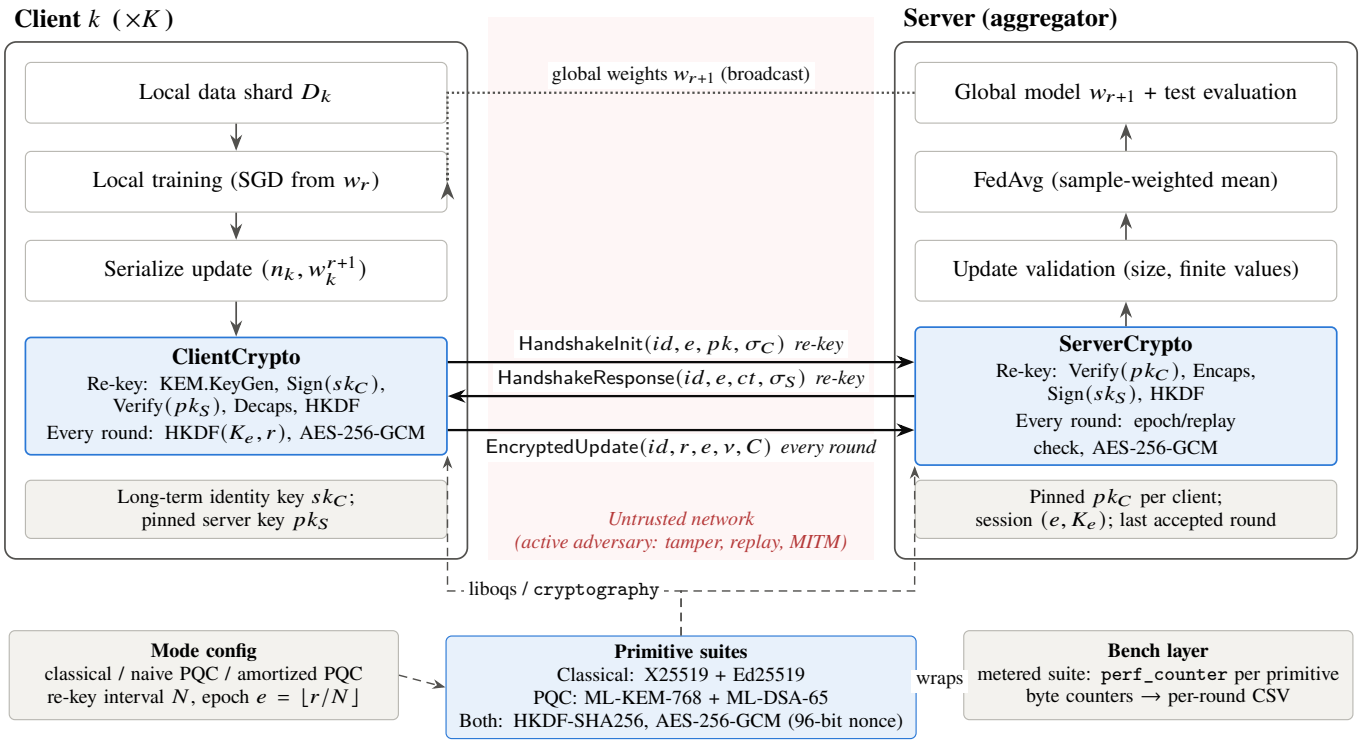


Fig. 1. System architecture.

TABLE I
[PLACEHOLDER: QUICK-PROFILE DATA] MAIN COMPARISON OF THE THREE MODES

Mode	N	HS	Crypto (ms)	Handshake (ms)	AES-GCM (ms)	HS KB/client	% wall	Acc. (%)
Classical	2	10	10.72 ± 0.45	4.63 ± 0.05	6.09 ± 0.40	0.4	0.251 ± 0.015	70.78 ± 2.62
Naive PQC	1	20	17.40 ± 0.08	11.08 ± 0.03	6.32 ± 0.06	34.7	0.408 ± 0.000	70.78 ± 2.62
Amortized PQC	2	10	11.87 ± 0.18	5.38 ± 0.37	6.49 ± 0.19	17.4	0.276 ± 0.007	70.78 ± 2.62

$K = 5$, 4 rounds, 2 seeds; times summed over all clients and the server; HS = handshakes.

TABLE II
[PLACEHOLDER: QUICK-PROFILE DATA] TIME PER PRIMITIVE OVER A SESSION (MS)

Mode	KeyGen	Encaps	Decaps	Sign	Verify
Classical	0.473 ± 0.000	0.570 ± 0.013	0.508 ± 0.015	1.053 ± 0.016	1.487 ± 0.052
Naive PQC	1.163 ± 0.004	0.470 ± 0.003	0.478 ± 0.007	5.320 ± 0.065	2.252 ± 0.059
Amortized PQC	0.563 ± 0.006	0.237 ± 0.000	0.239 ± 0.001	2.504 ± 0.357	1.141 ± 0.006

KEM keygen only; long-term identity keys are generated once at setup.

TABLE III
[PLACEHOLDER: QUICK-PROFILE DATA] AMORTIZED PQC VERSUS RE-KEY INTERVAL N

N	HS	Crypto (ms)	HS overhead/round (ms)	% of floor	HS KB/client
1	20	17.97 ± 0.32	2.867 ± 0.035	177.1	34.7
2	10	11.87 ± 0.18	1.344 ± 0.092	83.0	17.4
4	5	9.21 ± 0.05	0.693 ± 0.012	42.8	8.7
Naive PQC	20	17.40 ± 0.08	2.769	171.0	34.7

$K = 5$, 4 rounds, 2 seeds. Symmetric-only floor 1.619 ms/round; N^* = smallest N with handshake overhead per round $\leq$10% of the floor: not reached in the sweep.

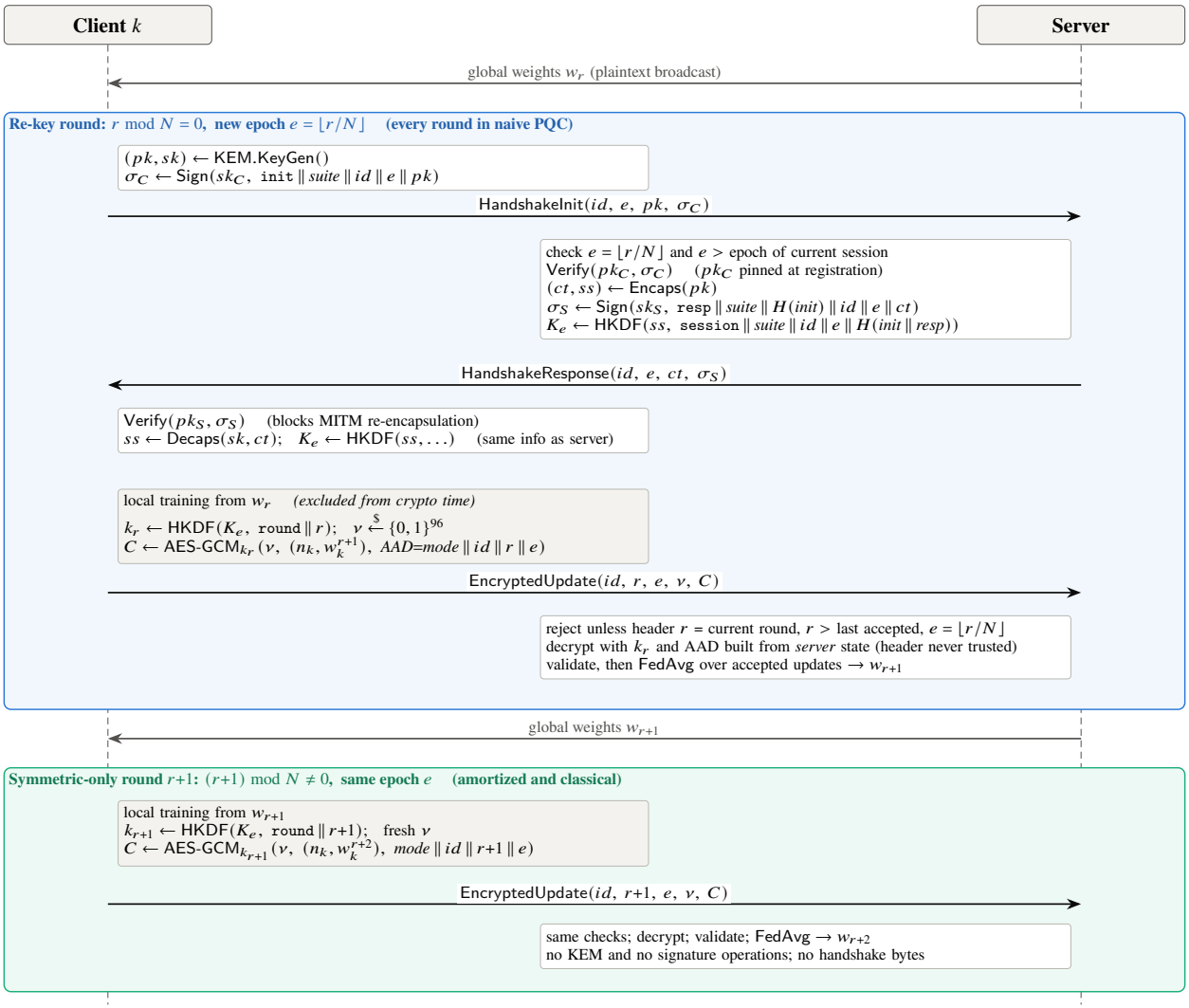


Fig. 2. Message sequence for a re-key round followed by a symmetric-only round.

TABLE IV
[PLACEHOLDER: QUICK-PROFILE DATA] CRYPTO COST VERSUS NUMBER OF CLIENTS K

K	Classical		Naive PQC		Amortized PQC	
	Crypto (ms)	% wall	Crypto (ms)	% wall	Crypto (ms)	% wall
3	6.96 ± 0.07	0.180 ± 0.002	10.88 ± 0.27	0.280 ± 0.015	7.42 ± 0.08	0.190 ± 0.002
5	10.72 ± 0.45	0.251 ± 0.015	17.40 ± 0.08	0.408 ± 0.000	11.87 ± 0.18	0.276 ± 0.007

$N = 2$ (Naive PQC: $N = 1$), 4 rounds, 2 seeds; times summed over all clients and the server.

TABLE V
[PLACEHOLDER: QUICK-PROFILE DATA] FINAL TEST ACCURACY (%) PER MODE AND SEED

Seed	Classical	Naive PQC	Amortized PQC	max $\lvert\Delta\rvert$ any round
0	68.93	68.93	68.93	0.00
1	72.63	72.63	72.63	0.00
Mean	70.78 ± 2.62	70.78 ± 2.62	70.78 ± 2.62	

TABLE VI
AUTOMATED TESTS (PYTEST); PARAMETRIZED CASES ARE COUNTED PER TEST

Test	Purpose	Result
Crypto layer (`tests/test_crypto.py`)		
`roundtrip_and_rekey_schedule`	Round trip over 7 rounds; handshake count follows the mode's re-key schedule	pass (4/4)
`nonces_unique_and_ciphertexts_differ`	50 encryptions of one plaintext use 50 distinct 96-bit nonces	pass (4/4)
`partial_participation_late_joiner`	Client joining late keys the epoch the schedule prescribes	pass (2/2)
`encrypt_without_session_raises`	Encryption is refused before a handshake	pass (4/4)
`tampered_ciphertext_rejected`	Bit flips in ciphertext, tag or nonce, truncation, client-ID swap fail the AEAD	pass (20/20)
`replayed_update_from_old_round_rejected`	Old-round update rejected verbatim, with rewritten header, and as a duplicate	pass (4/4)
`reordered_update_rejected`	Update for a future round is rejected	pass (3/3)
`rollback_to_earlier_round_rejected`	Fresh update for a round before the last accepted one is rejected	pass (1/1)
`old_epoch_key_rejected_after_rekey`	Update under the pre-re-key session key is rejected	pass (2/2)
`update_with_wrong_epoch_header_rejected`	Update whose header epoch disagrees with the schedule is rejected	pass (2/2)
`server_rejects_update_when_client_skipped_rekey`	Client that skipped a scheduled re-key is rejected	pass (2/2)
`handshake_for_wrong_epoch_rejected`	Handshake signed for a different epoch is rejected	pass (2/2)
`replayed_handshake_rejected`	Replayed old handshake cannot overwrite a live session	pass (2/2)
`impostor_signing_with_own_key_rejected`	Handshake signed with a key not registered for the claimed ID is rejected	pass (4/4)
`bitflipped_signature_rejected`	Corrupted or empty client signatures are rejected	pass (4/4)
`substituted_kem_key_rejected`	MITM substitution of the client's KEM public key is rejected	pass (4/4)
`signature_does_not_transfer_across_epochs`	A signature from one epoch cannot authorize another	pass (2/2)
`unknown_client_rejected`	Handshake from an unregistered client ID is rejected	pass (4/4)
`forged_server_response_rejected`	Client rejects a MITM re-encapsulated or re-signed server response	pass (4/4)
`invalid_config_rejected`	Invalid mode / re-key interval combinations are rejected	pass (4/4)
`config_accepts_mode_strings`	Modes can be given as strings; naive PQC fixes $N = 1$	pass (1/1)
`classical_rekey_interval_accepted`	Classical mode accepts key-once or a re-key interval	pass (1/1)
`server_signs_response_in_every_mode`	Server response is signed (Ed25519 / ML-DSA-65) in every mode	pass (4/4)
`wire_sizes`	Key, ciphertext and signature sizes match FIPS 203/204; 16-byte GCM tag	pass (1/1)
FL harness (`tests/test_fl.py`)		
`flat_params_roundtrip`	Model parameters survive flatten / restore; size mismatch is rejected	pass (1/1)
`update_serialization_roundtrip_and_validation`	Update (de)serialization round trip; wrong size, truncation, NaN rejected	pass (1/1)
`fedavg_is_sample_weighted`	FedAvg computes the sample-weighted mean	pass (1/1)
`splits_partition_the_data`	IID and Dirichlet splits are disjoint and cover the dataset	pass (2/2)
`metered_suite_times_ops_and_interoperates`	Instrumented suite times each primitive and derives identical keys	pass (1/1)
`harness_rekey_schedule_and_csv`	Harness handshake schedule, CSV schema and per-component byte counts	pass (4/4)
`harness_is_deterministic_and_mode_independent`	Same seed gives identical accuracy in every mode	pass (1/1)
`harness_drops_tampered_and_replayed_updates`	Tampered and replayed updates are excluded from aggregation	pass (1/1)
`harness_keeps_global_model_when_all_updates_rejected`	If every update is rejected the global model is unchanged	pass (1/1)

98/98 test cases passed.

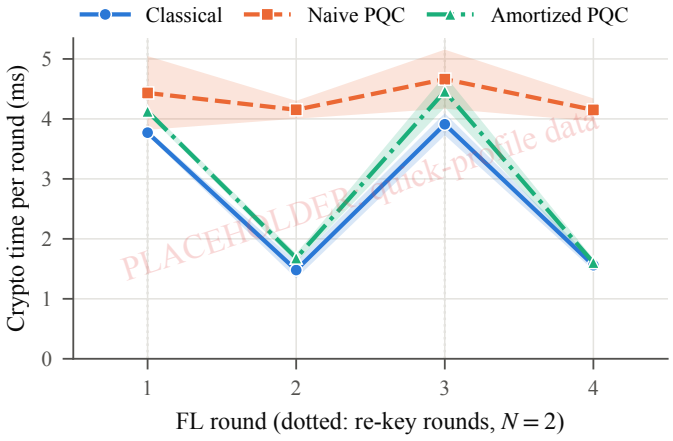


Fig. 3. Crypto time per round by mode (mean ± std over seeds).

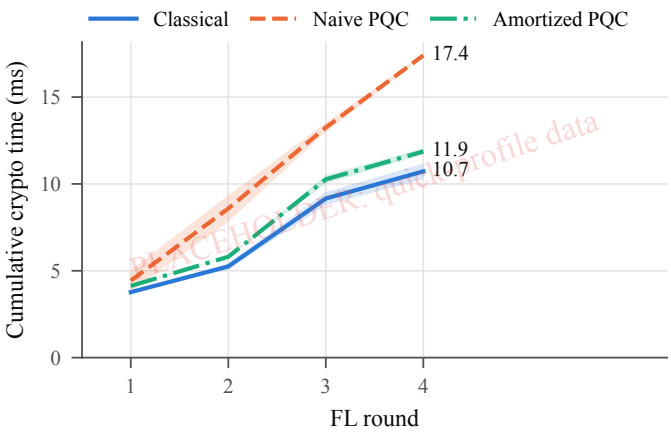


Fig. 4. Cumulative crypto time over rounds by mode.

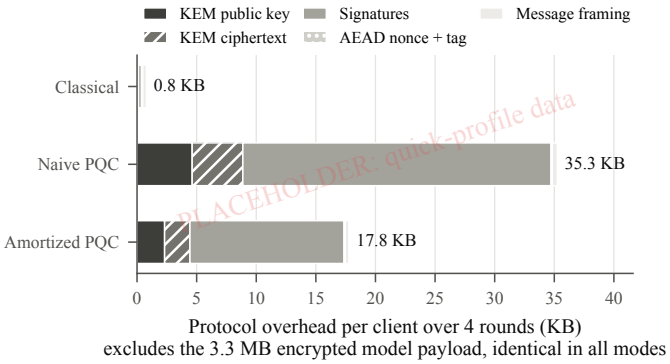


Fig. 5. Protocol bytes on the wire per client, by component.

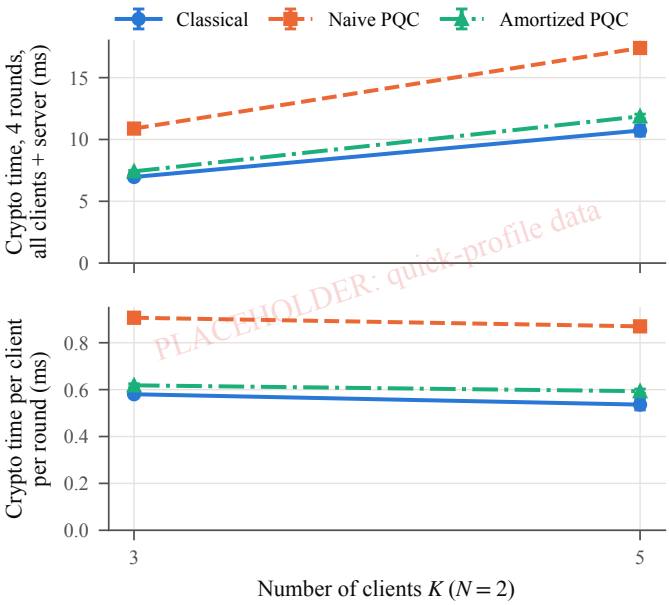


Fig. 7. Crypto overhead versus number of clients K.

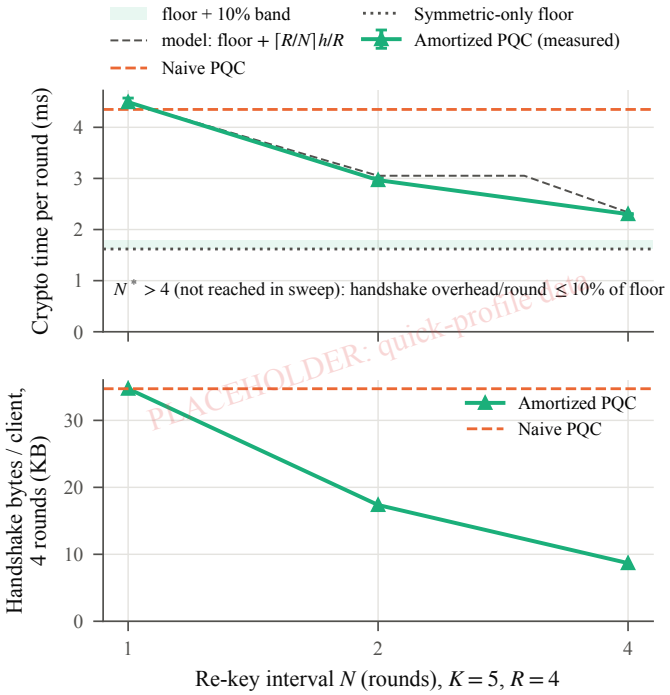


Fig. 6. Overhead versus re-key interval N (amortized PQC). N^*: smallest N whose handshake overhead per round is within 10% of the symmetric-only floor (shaded band).

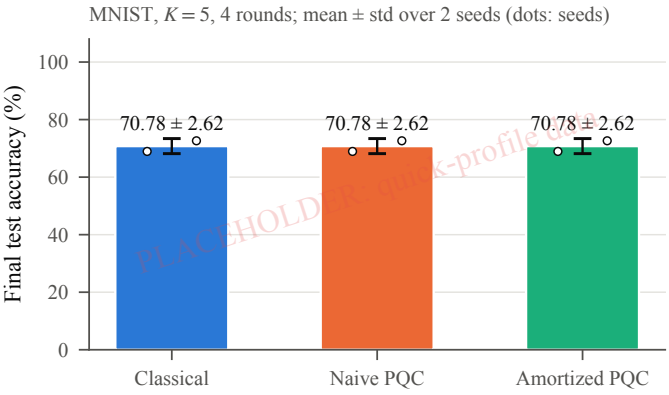


Fig. 8. Final test accuracy per mode (mean ± std over seeds).